Design and Implementation of a Trading Strategy Simulator Based on Real-Time Market Data

MSc Risk Management & Financial Engineering

Imperial College London

CID:06029495

Project Type: Software Report

Word Count: 2839

Contents

1 Introduction 2

2 Related Work 4

3 System Design 5

4 Implementation 7

5 Empirical Case Study: SOLUSDT 12

6 Strategy Comparison: Stoikov vs NaiveMM 14

7 User Interface for Strategy Simulation 18

8 Conclusion 23

References 24

1 Introduction

Market making is a key component of modern electronic markets, providing liquidity, reducing bid-ask spreads and facilitating efficient trading. As noted by Li et al. (2025, p.1) *"We demonstrate that market makers contribute to stock liquidity in hybrid markets characterized by a high probability of asset value jumps or a low arrival rate of market order traders"*. his application project is a compact modular simulator that tests the quoting policy on real Binance Level-2 (L2) order book data. The paper compares the risk-adjusted Avellaneda-Stoikov (AS) model with a simple heuristic framework (naive market maker, NMM), where we focused on implementation, calibration and decision support.

Client Specification

Client: Head of Electronic Market Making, Digital Assets Desk.
Business question: For large-cap crypto pairs on Binance which approach: AS or NMM should be the default and which parameter ranges should be considered to execute frictions and inventory limits ?

Scope

- **Assets:** `BTCUSDT`, `ETHUSDT`, `SOLUSDT` .
- **Data:** Historical Binance L2 snapshots with replay step-by-step , computed mid, spread, top-of-book sizes and imbalance.
- **Strategies:** AS with γ (risk aversion) and k (intensity); NMM with base spread and volatility multipliers.
- **Execution:** Probabilistic fills with slippage, order size caps and hard inventory bounds.

Deliverables

1. Runnable tool (CLI + Streamlit UI) to configs, run code and visualise experiments on L2 data.
2. Side-by-side evaluation on multiple assets and parameter sets, with a clear recommendation (when to prefer AS vs NMM).
3. Reproducible artefacts per run (CSV metrics, PNG charts, config + seed).

Acceptance criteria

- Deterministic replay (fixed seeds); runs for ∼2,000 snapshots complete within minutes.
- Metrics: final cumulative PnL, Sharpe Ratio, Sortino Ratio, maximum drawdown (USDT), inventory standard deviation, average trade size, total number of trades, fill rate (%), turnover and average holding period.
- Documented calibration for γ, k (AS) and spreads (NMM),sensitivity analysis.

Constraints

- No live trading or exchange connectivity.
- Need to model queue position, latency and hidden liquidity in reduced form.
- Fixed evaluation windows to keep results auditable and comparable.

Main output: The sofware (simulator + Streamlit UI). This report is describing the tool and showing results. All main modules, functions and scripts what used for replay data, logic of strategy, execution modeling and metrics calculation are fully done in Python and are in the Appendix. We run controlled tests with different assets and parameter grids to understand PnL-risk trade-off, fill dynamic and how inventory is used, also we give calibrated and repeatable recommendation for the desk.0

2 Related Work

Market making plays a fundamental role in electronic markets. In our work, we emphasize simple yet practical components that serve as the bridge between theoretical frameworks and real-world data. This is all within the scope of an L2-based simulator, where we explore and compare the Stoikov model against a NaiveMM approach.

Theoretical Models

The Avellaneda–Stoikov model (Avellaneda and Stoikov, 2008) treats quoting as expected-utility maximization with inventory risk. As Falces Marín et al. (2022, p.3) write, *"The successive orders generated by this procedure maximize the expected exponential utility of the trader's profit and loss (P&L) profile at a future time, T (usually, the daily closing time for trade), for a given level of agent inventory risk aversion"*. We implement this objective as our theory-driven benchmark and compare it to a simple heuristic (NaiveMM), which follows ideas on execution costs and risk control from Cartea, Jaimungal and Penalva (2015).

Empirical Market Microstructure

We replay real Binance L2 states and model probabilistic fills and slippage to capture observed frictions from Bouchaud , Farmer and Lillo (2008, p.5) refer to *'incremental execution of large hidden trading orders"* and to order-flow long-memory coexisting with (statistical) efficiency, which together drive *dynamic adjustments of liquidity that are strongly asymmetric between buyers and sellers"*. By analogy to crypto order books, we incorporate asymmetric liquidity responses, inventory-aware quoting and execution which is not without friction.

Tools and Positioning

Portfolio backtesters (e.g., Zipline, Backtrader) mostly ignore L2 mechanics, while agent/message-level simulators (e.g., ABIDES; ML toolkits like Qlib) are heavier to configure. We take the middle ground: a lightweight L2 simulator over Binance snapshots with realistic but tractable execution, side-by-side *Stoikov* vs. *NaiveMM* and concise metrics (Final P&L, Sharpe/Sortino from P&L increments, Max Drawdown, fill rate, turnover, inventory stats) via an interactive UI. sifn

3 System Design

The simulator is split into small, focused modules. Each module is designed to do one specific job effectively and do communications with others through a clear and narrowly scoped interface. This keeps the code easy to extend and audit.

Core Components

- **Data Replay (BinanceL2ReplaySim)** — Step–accurate replay of Binance L2 snapshots (CSV/Parquet) at a configurable frequency, exposes the current top-of-book, mid, spread and depth to strategies.
- **Strategy Module (StoikovStrategy / NaiveMMStrategy)** — Generates bid/ask quotes based on the current market state. Stoikov incorporates inventory, risk aversion γ and volatility estimate σ to skew quotes; NaiveMM applies a fixed base spread with optional volatility adjustment.
- **Execution Model** — Reduced-form matching against the replayed book with probabilistic fills, partial fills and slippage.Configurable knob: queue position, latency and hidden liquidity.
- **Accounting & Metrics** — Mark-to-mid P&L and inventory updates on every step. We track: cumulative P&L, Sharpe/Sortino (from P&L diffs), max drawdown, fill rate, turnover, inventory stdev, average trade size and number of trades. Metrics can be exported to CSV.
- **User Interface (Streamlit)** — Run controls, parameter panels and side-by-side strategy comparison with live plots (P&L, inventory, fills) and after that via one-click we can export figures and tables.

Workflow

1. Load and time-align L2 snapshots in the replay engine.
2. Feed the current market state to the chosen strategy.
3. Strategy returns quotes; orders are sent to the execution model.
4. Execution determines fills, applies slippage and updates cash/inventory.
5. the metrics module updates statistics with accounting records state.
6. The UI renders results and on request saves CSV/PNG artifacts.

Extensibility

- Swap the data layer for another source (e.g., live feed) without touching strategy code.
- Easily replace execution logic to test alternative impact/latency/queue models.
- Extend the metrics set or wire batch runners for multi-asset experiments.

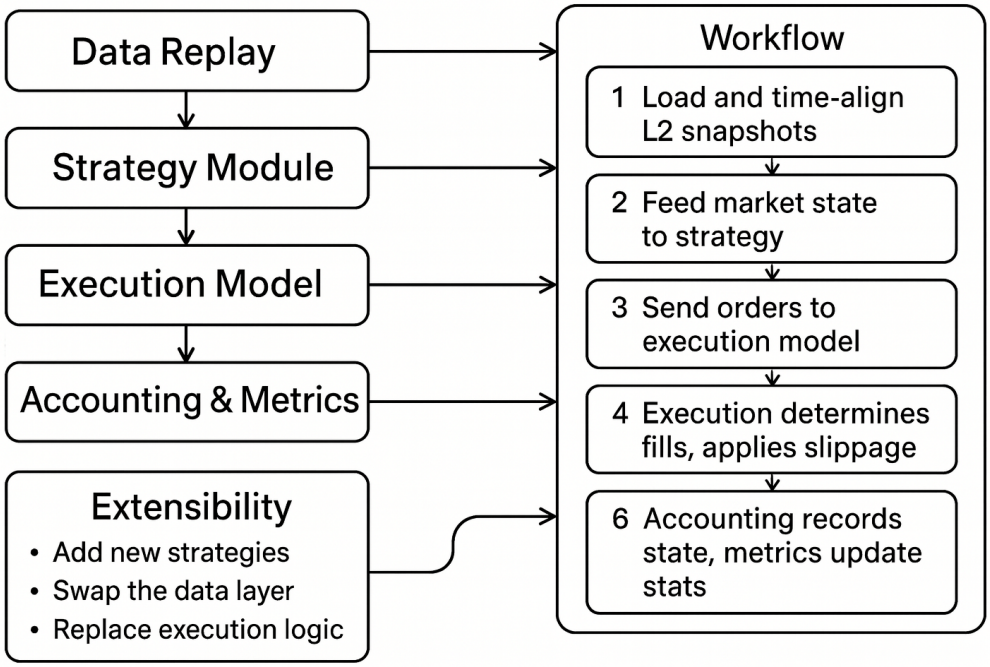


Figure 1: Architecture of the market-making simulator.

4 Implementation

The simulator is made in Python with the addition of CSS modules and use modular design. It separate into: (i) replay of data, (ii) logic for strategy, (iii) execution and fills, (iv) tracking of performance and (v) user interface. Every part give small and testable interface.

Notation and Units

Symbol	Description
m_t	Mid-price at step t (quote per asset).
$S^b_{t,i}, V^b_{t,i}$	Bid price and visible size at depth $i = 1, \dots, L$.
$S^a_{t,i}, V^a_{t,i}$	Ask price and visible size at depth $i = 1, \dots, L$.
L	Number of visible book levels per side in each snapshot.
Δt	Time between snapshots (seconds).
W	Rolling window length (snapshots) for returns/volatility.
A	Time–scaling factor to daily units (e.g., $A \approx 86400/\Delta t$).
q_t	Inventory (asset units) after step t.
$\hat{\sigma}_t$	Realised volatility estimate at t (per $\sqrt{\text{day}}$).
T	Strategy horizon (seconds), $t \in [0, T]$.
γ	Risk aversion (dimensionless, $\gamma > 0$).
k	Exponential arrival slope in $\lambda(\delta) = A\, e^{-k\delta}$ (dimensionless, $k > 0$).
λ_{inv}	Inventory penalty coefficient in the spread (dimensionless, ≥ 0).
$\rho_t \in [0, 1]$	Queue-position ratio at placement (0 = back of queue, 1 = front).
$f_{\text{maker}}, f_{\text{taker}}$	Maker and taker fee rates (fraction of notional).
`tick_size`	Instrument tick size (quote).
δ_{ticks}	Distance-to-mid (in ticks) of our quote.
$V^{\text{vis}}_{t,\cdot}$	Visible size at our book level at time t.
x_t	Our order size at step t (asset units).
$b_t \in [-1, 1]$	Short-horizon direction proxy (e.g., signed imbalance/micro-price).
s_{base}	Baseline spread (ticks) in the Naive MM.
c_{vol}	Volatility multiplier in the Naive MM.
VolProxy_t	Volatility proxy (e.g., $\lvert\Delta m_t\rvert$, $\hat{\sigma}_t$, ATR).
ATR	Average True Range proxy (ticks or quote units).
$\alpha_0, \dots, \alpha_4$	Coefficients in the logistic fill model.
`slip_ticks`	Slippage at fill (ticks).
E_t	Equity in quote currency at time t.
D	Number of trading days in the evaluation sample.
$Q_{\max}$	Inventory limit threshold for violation counts.

Table 1: Notation and units used throughout the implementation.

Step 1 — Returns and Volatility. Following Avellaneda and Stoikov (2008) and Cartea et al. (2015), we define snapshot log-returns over a rolling window W snapshots

as:

$$r_\tau = \log m_\tau - \log m_{\tau-1}, \qquad \tau = t - W + 1, \ldots, t, \tag{1}$$

$$\bar{r}_t = \frac{1}{W} \sum_{\tau=t-W+1}^{t} r_\tau, \tag{2}$$

$$\hat{\sigma}_t = \sqrt{\frac{A}{W-1} \sum_{\tau=t-W+1}^{t} \big(r_\tau - \bar{r}_t\big)^2}. \tag{3}$$

Here A rescales the variance from the snapshot frequency to a per-$\sqrt{\text{day}}$ basis. For fixed-frequency snapshots, we stated $A \approx \frac{86400}{\Delta t}$, where Δt is the time step in seconds between snapshots. This annualisation ensures $\hat{\sigma}_t$ is comparable to daily volatility estimates and is using in market-making models.

Data Replay Engine

Class: `BinanceL2ReplaySim`. Input schema `timestamp, side, price, size, level`. Each snapshot provides L bid and L ask levels; state:

$$\mathcal{M}_t = \{(S^b_{t,i}, V^b_{t,i})_{i=1}^{L},\ (S^a_{t,i}, V^a_{t,i})_{i=1}^{L}\}, \tag{4}$$

$$m_t = \tfrac{1}{2}\big(S^a_{t,1} + S^b_{t,1}\big). \tag{5}$$

Features: step-accurate replay, deterministic iteration, pluggable live adapters.

Strategy Module

Two strategies are implemented: Avellaneda–Stoikov (`stoikov.py`) and Naive Market Maker (`naive_mm.py`) with execution handled by `ExecutionModel` (`execution.py`). They are run via `main.py`, which loads data, estimates volatility, runs the strategy loop and computes metrics (see in the Appendix).

Step 2 — Avellaneda–Stoikov (inventory-aware). Under exponential arrival intensities $\lambda(\delta) = Ae^{-k\delta}$, the reservation price and the optimal half-spread:

$$r_t = m_t - q_t\, \gamma\, \hat{\sigma}_t^2\, (T - t), \tag{6}$$

$$\delta^\star_t = \tfrac{1}{2}\gamma\, \hat{\sigma}_t^2\, (T - t) + \frac{1}{\gamma} \log\Big(1 + \frac{\gamma}{k}\Big), \tag{7}$$

$$p_t^{\text{ask}} = r_t + \delta^\star_t, \qquad p_t^{\text{bid}} = r_t - \delta^\star_t. \tag{8}$$

An optional inventory-smoothing term $\delta'_t = \delta^\star_t + \lambda_{\text{inv}} q_t^2$ is applied to accelerate mean reversion of q_t. Everything implemented in `StoikovStrategy` (`stoikov.py`).

Step 3 — Naive Market Maker (symmetric baseline). Following the *symmetric baseline* in Avellaneda–Stoikov (2008, §3.3), we use the same $\delta^\star_t$ but center quotes on the mid-price m_t without inventory adjustment:

$$s_t = 2\,\delta^\star_t, \quad p_t^{\text{ask}} = m_t + \tfrac{1}{2}s_t, \quad p_t^{\text{bid}} = m_t - \tfrac{1}{2}s_t. \tag{9}$$

Optionally s_t scales with a short-term volatility proxy. Implemented in `NaiveMMStrategy` (`naive_mm.py`).

Execution. Both strategies interact with the `ExecutionModel` (`execution.py`) to simulate matching of limit orders with Level-2 updates.

Execution Model

Class: `ExecutionModel`. Goal: decide fills given $\mathcal{M}_t$ and quotes.

Step 4 — Fill probability (queue & distance). With queue ratio ρ_t, distance δ_{ticks} and visible size $V^{\text{vis}}_{t,\cdot}$ at our level:

$$p_{\text{fill}} = \sigma\Big(\alpha_0 + \alpha_1 \rho_t - \alpha_2\, \delta_{\text{ticks}} + \alpha_3\, \min\big(1, \tfrac{x_t}{V^{\text{vis}}_{t,\cdot}}\big) + \alpha_4 b_t\Big), \tag{10}$$
$$\sigma(x) = \frac{1}{1+e^{-x}}.$$

$b_t \in [-1, 1]$ is a short-horizon direction proxy (e.g., signed imbalance). This reduced-form is consistent with the exponential decay of arrivals in A–S.

Step 5 — Slippage & fees. execution price

$$\tilde{p}_t = p_t \pm \big(\text{slip_ticks} \cdot \text{tick_size}\big), \tag{11}$$

and cash updates apply maker/taker fees $f_{\text{maker/taker}}$.

Step 6 — State update & PnL. With buy size x_t^+ at $\tilde{p}_t^+$ and sell size x_t^- at $\tilde{p}_t^-$:

$$q_{t+1} = q_t + x_t^+ - x_t^-, \tag{12}$$
$$\text{cash}_{t+1} = \text{cash}_t - x_t^+\, \tilde{p}_t^+ (1+f) + x_t^-\, \tilde{p}_t^- (1-f), \tag{13}$$
$$E_t = \text{cash}_t + q_t\, m_t, \qquad \text{(equity in quote currency)} \tag{14}$$
$$\Delta\text{PnL}_t = E_t - E_{t-1}, \qquad \text{PnL}_t = E_t - E_0. \tag{15}$$

Metrics Module (exactly as used in experiments)

The Python implementation of these metrics is provided in `evaluate_metrics.py` (see Appendix).

Step 7 — Cumulative PnL. The final cumulative profit and loss (P&L) is computed :

$$\text{PnL}_{\text{final}} = E_T - E_0, \tag{16}$$

where E_t is portfolio equity at time t.

Step 8 — Daily aggregation. Intraday P&L changes ΔPnL_t are aggregated to daily returns:

$$r_d = \sum_{t \in d} \Delta\text{PnL}_t. \tag{17}$$

Step 9 — Sharpe ratio (daily). Following Sharpe (1994):

$$\bar{r} = \frac{1}{D}\sum_{d=1}^{D} r_d, \qquad s = \sqrt{\frac{1}{D-1}\sum_{d=1}^{D}(r_d - \bar{r})^2}, \tag{18}$$

$$\text{Sharpe} = \frac{\bar{r}}{s}\sqrt{252}. \tag{19}$$

Step 10 — Sortino ratio (daily). Using the downside deviation s^- as in Sortino and Price (1994):

$$s^- = \sqrt{\frac{1}{D-1}\sum_{d=1}^{D}\min(r_d, 0)^2}, \tag{20}$$

$$\text{Sortino} = \frac{\bar{r}}{s^-}\sqrt{252}. \tag{21}$$

Step 11 — Maximum drawdown. As in Magdon-Ismail et al. (2004), define $E_t^{\max} = \max_{u \le t} E_u$ and $DD_t = E_t^{\max} - E_t$:

$$\text{MDD} = \max_t DD_t. \tag{22}$$

Step 12 — Fill rate (orders & size). Order execution efficiency, following the definition as in Rule 605 (SEC, 2000) :

$$\text{FillRate}_{\text{orders}} = \frac{N_{\text{filled}}}{N_{\text{sub}}}, \qquad \text{FillRate}_{\text{size}} = \frac{S_{\text{fill}}}{S_{\text{req}}}, \tag{23}$$

Step 13 — Turnover (size & notional). Trading turnover is defined as in Qian et al. (2007) :

$$\text{Turnover}_{\text{size}} = \sum_t \big(x_t^+ + x_t^-\big), \quad \text{Turnover}_{\text{notional}} = \sum_t \big(x_t^+ \tilde{p}_t^+ + x_t^- \tilde{p}_t^-\big), \tag{24}$$

Step 14 — Inventory dispersion & violations. Inventory risk metrics follow the framework:

$$\text{InventoryStd} = \sqrt{\frac{1}{T-1}\sum_{t=1}^{T}(q_t - \bar{q})^2}, \qquad \text{Violations}(Q_{\max}) = \#\{t : \ |q_t| > Q_{\max}\}, \tag{25}$$

User Interface

The implementation is described in detail in Section 7 (*User Interface for Strategy Simulation*), with screenshots and usage examples. The Streamlit application is structured into three modules (`app.py`, `logic.py`, `ui.py`), enabling clean separation of presentation, control logic and parameter handling. Full source code also provided in Appendix.

Extensibility and Assumptions

Each strategy follows a unified interface designed to generate bid/ask quotes based on the current market state.In our implementation, this logic resides in:

- `StoikovStrategy` (in `stoikov.py`)
- `NaiveMMStrategy` (in `naive_mm.py`)
- `ExecutionModel` (in `execution.py`)

The data playback module can be replaced with an online adapter without changing the strategy code. The fill model (10) can be replaced by an exchange-specific queue simulator when prints/message data are available.

Recommended Parameter Ranges and Constants

Parameter calibration is based on empirical Binance ETHUSDT/BTCUSDT/SOLUSDT Level-2 data, sampled at $\Delta t \approx 0.02\,\text{s}$ (50 Hz) with depth $L = 10$, collected via the public Binance API using `fetch_binance_l2_snapshot()` from `utils/binance_fetch.py` (see Appendix). Ranges follow standard modelling in Avellaneda and Stoikov (2008) and Cartea et al. (2015).

Parameter	**Symbol**	**Typical Range**	**Notes**
Risk aversion	γ	0.01–0.1	Higher $\gamma \Rightarrow$ stronger inventory control
Arrival slope	k	0.5–2.0	$\lambda(\delta) = Ae^{-k\delta}$
Inventory penalty	λ_{inv}	0–0.005	Quadratic spread widening in $\lvert q\rvert$
Order size	x_t	0.01–0.1	Asset units (e.g., ETH, BTC)
Base spread	s_{base}	2–6	Ticks; NaiveMM only
Volatility multiplier	c_{vol}	0.5–2.0	NaiveMM only
Volatility window	W	500–2000	$\approx$10–40 s at 50 Hz
Slippage	–	0–2	Ticks (deterministic)
Maker fee	f_{maker}	-0.0002	Binance spot, fraction of notional
Taker fee	f_{taker}	0.0004	Binance spot, fraction of notional
Annualisation (Sharpe)	–	252	For daily r_d; else see (3)

Table 2: Parameter ranges and constants for backtests (Binance L2 data).

5 Empirical Case Study: SOLUSDT

This case study evaluates the simulator and `StoikovStrategy` performance using Level-2 order book data for the **SOL/USDT** trading pair from Binance.

Test Setup

- **Data source:** Binance L2 historical snapshots.
- **Sample size:** 1,000 sequential snapshots.
- **Replay method:** `BinanceL2ReplaySim` with one-snapshot time steps.
- **Execution model:** Custom Python implementation (`execution.py`, which use probabilistic fill logic based on queue position, price distance and order size and which also includes slippage simulation.
- **Evaluation:** Strategy performance metrics calculated via `evaluate_metrics.py` with Sharpe/Sortino ratios, max drawdown, fill rate, turnover and average holding period.
- **Implementation:** Executed using the `case_study_stoikov.py`, which automates data loading, playback, strategy launch, and metric generation.

All function implementations for the modules mentioned above are listed in the Appendix.

Test Parameters

- Risk aversion $\gamma = 0.1$
- Market depth $k = 1.2$
- σ = realized volatility from log-returns
- Inventory limits: $[-5, +5]$
- Order size = 1.0 unit

Results

Table 3: Performance Metrics — Stoikov Strategy on SOL/USDT

Metric	**Value**
Final PnL	53.79 USDT
Sharpe Ratio	1.22
Sortino Ratio	0.52
Max Drawdown	25.44 USDT
Inventory Std	2.00
Average Trade Size	1.02
Number of Trades	169

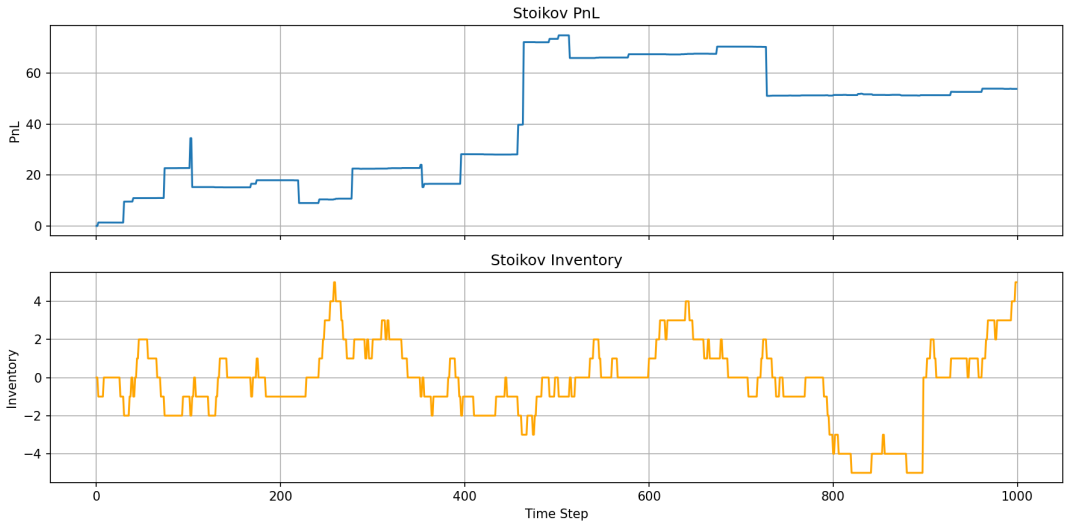


Figure 2: PnL and Inventory Trajectories — Stoikov Strategy on SOL/USDT

Discussion

The strategy maintained inventory within limits and achieved a positive PnL, with a Sharpe Ratio above 1 indicating stable returns. Drawdowns were moderate (max: 25.44 USDT), which shows resilience to short-term adverse moves. These results confirm that Stoikov's logic works as intended and that the simulator can reproduce realistic market conditions for controlled testing. Full code implementations of the execution model and metrics are provided in the Appendix.

6 Strategy Comparison: Stoikov vs NaiveMM

This section compares the performance of the **Stoikov** and **Naive Market Maker (NaiveMM)** strategies on two trading pairs: `BTCUSDT` and `SOLUSDT` where used Binance Level-2 order book data. Both strategies were evaluated over **2000 sequential snapshots** per pair, employing a probabilistic execution model with slippage implemented in `main.py` (function `compute_pnl`) and the common run-loop logic of each strategy.

The experiment setup and batch processing for multiple symbols were automated via `scripts/batch_compare.py`, which calls `main.py` for each symbol and metrics tables using `evaluate_strategy_metrics()` from `scripts/evaluate_metrics.py`.

BTCUSDT Results

Figures 3 and 4 present the inventory and PnL trajectories. Stoikov shows a steeper inventory curve and higher turnover compared to NaiveMM, indicating more frequent position changes and a higher trading intensity. This increased activity is associated with a higher fill rate, but also with a larger maximum drawdown.

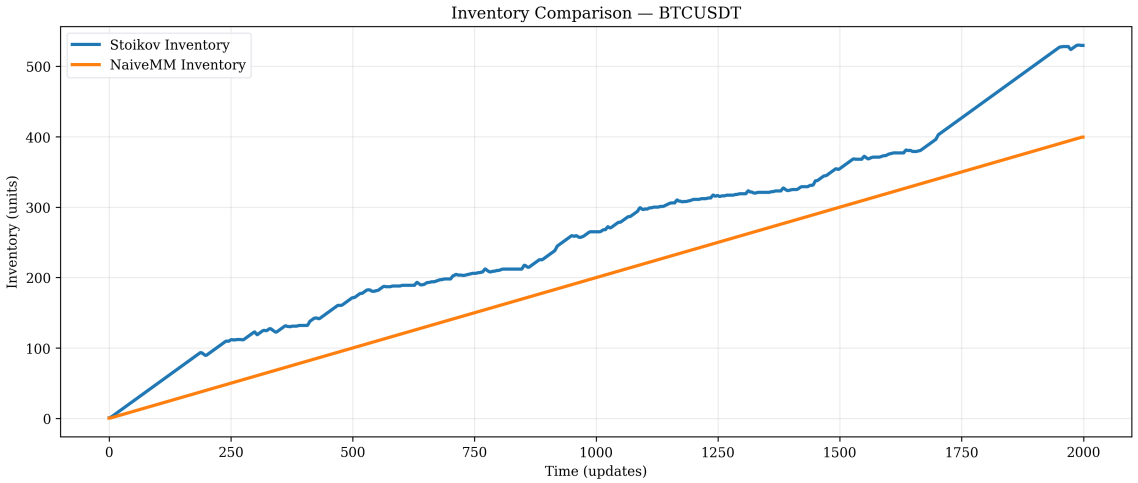


Figure 3: Inventory comparison — BTCUSDT

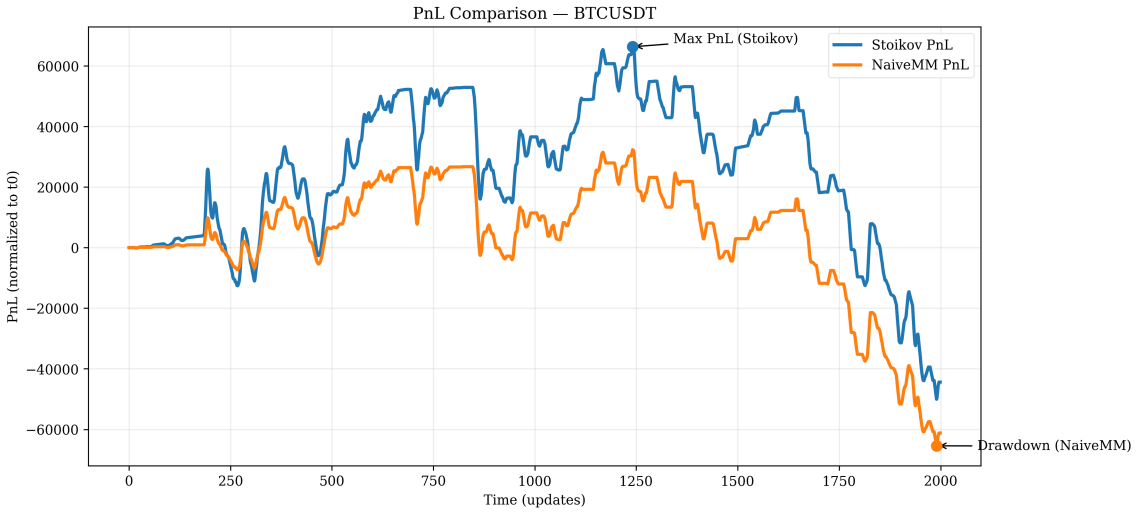


Figure 4: PnL comparison — BTCUSDT

Table 4: Performance comparison — BTCUSDT (2000 snapshots)

Strategy	Sharpe	Max Drawdown	Fill Rate	Turnover	Avg Hol Period
Stoikov	-0.228	120170.00	153.57	760.0	2.91
NaiveMM	-0.427	100712.00	100.00	399.8	1.00

Analysis:

- Higher turnover in Stoikov (760.0 vs 399.8) corresponds to a 53% higher fill rate, suggesting that more aggressive quoting leads to more matched orders.
- The higher activity is associated with a deeper maximum drawdown (120,170 USDT vs 100,712 USDT), indicating increased exposure to adverse price moves.
- The less negative Sharpe ratio of Stoikov implies a smaller loss per unit of volatility compared to NaiveMM under these market conditions.

SOLUSDT Results

Figures 5 and 6 show the inventory and PnL for SOLUSDT. NaiveMM demonstrates a shorter average holding period and lower turnover, while Stoikov exhibits longer holding periods and higher turnover.

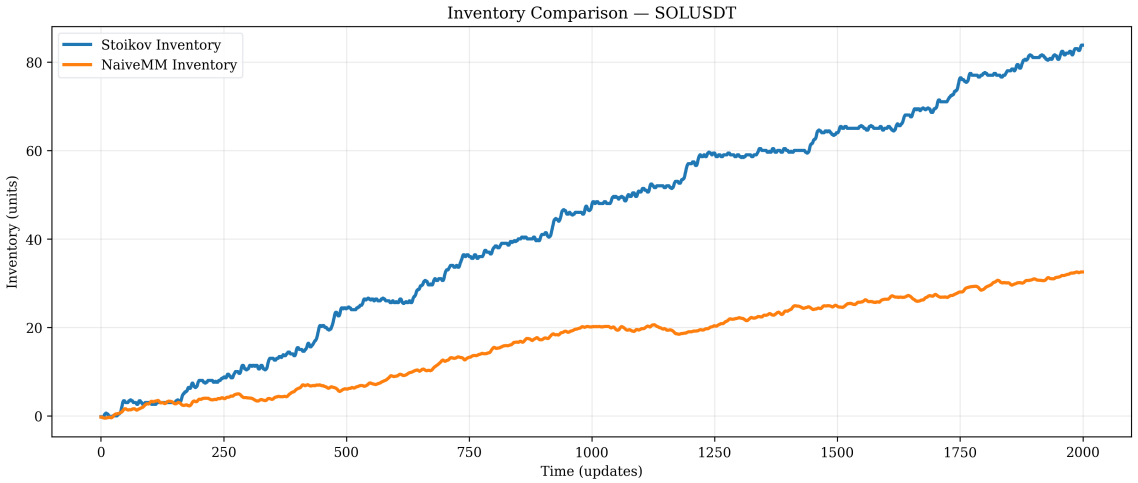


Figure 5: Inventory comparison — SOLUSDT

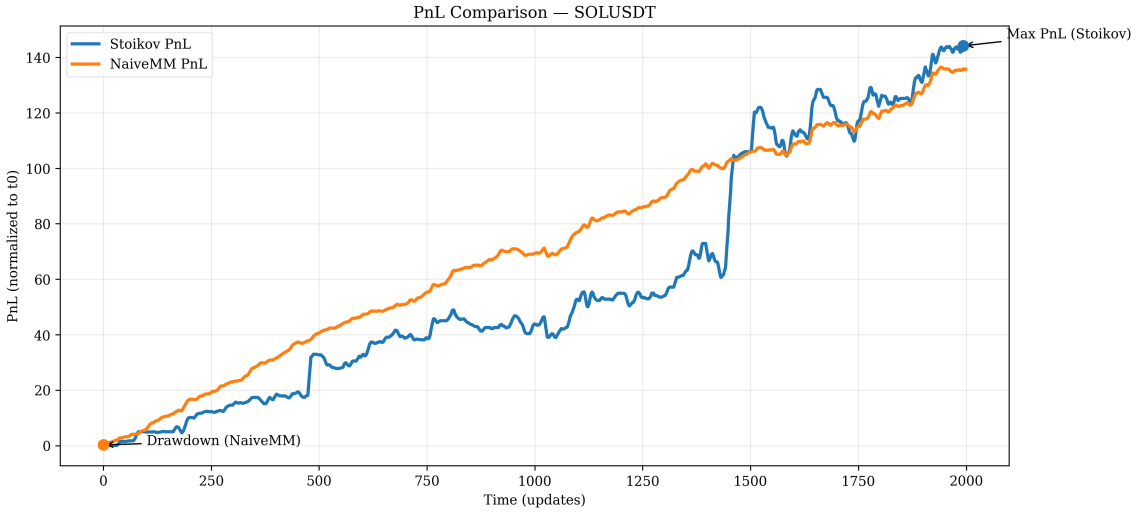


Figure 6: PnL comparison — SOLUSDT

Table 5: Performance comparison — SOLUSDT (2000 snapshots)

Strategy	Sharpe	Max Drawdown	Fill Rate	Turnover	Avg Hol Period
Stoikov	1.105	19.84	38.64	534.0	3.74
NaiveMM	3.296	4.21	45.60	182.4	2.19

Analysis:

- NaiveMM achieves a higher Sharpe ratio (3.296 vs 1.105) and lower maximum drawdown, consistent with lower turnover and reduced market exposure.
- Stoikov's turnover is nearly three times higher, with a longer average holding period, suggesting a more continuous quoting presence.

- The lower fill rate of Stoikov (38.64% vs 45.60%) despite higher turnover may indicate that quotes are placed further from the mid-price or are more frequently updated before execution.

Summary Across Pairs

For BTCUSDT, higher turnover and Stoikov fill ratio coincide with deeper drawdowns and a less negative Sharpe ratio, which suggests that aggressive quoting increases the frequency of trades, but also increases the exposure to volatility. As for SOLUSDT, NaiveMM provides higher risk-adjusted returns and lower drawdowns, while Stoikov supports higher market engagement through increased turnover and longer retention periods. For both pairs, the differences in growth, drawdown and fill rate reflect a trade-off between trading intensity and exposure to adverse price fluctuations.

7 User Interface for Strategy Simulation

The simulator includes an interactive `Streamlit`-based web interface, which allows users to configure, launch and analyze marketing strategies without changing the source code. The UI is implemented across `app.py` (top-level app and controls), `logic.py` (orchestration via `run_strategy()`), and `ui.py` (rendering via plots and metrics). Inventory and PnL are reconstructed with `compute_inventory_pnl()` in `logic.py`, while metrics are computed by `evaluate_strategy_metrics()` from `scripts/evaluate_metrics.py`. Data replay is handled by `BinanceL2ReplaySim` and collected by `collect_if_needed()`.

Purpose

The interface provides an accessible front-end for:

- Selecting assets and strategies (controls in `app.py`).
- Adjusting simulation parameters (sidebar controls in `app.py`).
- Launching simulations and viewing real-time results (triggering `run_strategy()` in `logic.py`; live plots via `ui.py`).
- Exporting performance data for offline analysis (CSV via `download_metrics_csv()`).

Features

- **Strategy selection:** Stoikov, NaiveMM or both (side-by-side comparison, wired in `app.py` → `run_strategy()`).
- **Asset selection:** Default `ETHUSDT`, extensible to other symbols (e.g., `BTCUSDT`, `SOLUSDT` symbol handling in `app.py`.
- **Parameter control:**
  - `Stoikov`: risk aversion (γ), market depth parameter (k), order size (passed to `StoikovStrategy` in `logic.py`).
  - `NaiveMM`: base spread, volatility multiplier, order size (passed to `NaiveMMStrategy` in `logic.py`).
- **Debug mode:** Optional verbose trade logs (toggle in `app.py` propagated into `run_strategy()`).

Workflow

1. Select the trading pair and strategy to run (`app.py`).
2. Adjust strategy parameters via the sidebar controls (b `StoikovStrategy`/ `NaiveMMStrategy`).
3. Launch the simulation: `app.py` calls `run_strategy()` (in `logic.py`), which:
   - loads/replays data with `BinanceL2ReplaySim`,

- estimates volatility via `compute_sigma_from_mid()`,
- runs strategies and reconstructs paths using `compute_inventory_pnl()`.

4. The interface streams PnL and inventory updates in real time and plots are rendered by `show_pnl_plot()` and `show_inventory_plot()` (`ui.py`).

5. Upon completion, view calculated performance metrics computed by `evaluate_strategy_metrics()` (`scripts/evaluate_metrics.py`) and displayed via `show_metrics()`.

Outputs

- **Plots:** Time series of PnL and inventory trajectories (`show_pnl_plot()`, `show_inventory_plot`); includes smoothing and drawdown shading.
- **Performance metrics:** Sharpe ratio, max drawdown, fill rate, turnover and average holding period (`evaluate_strategy_metrics()` → `show_metrics()`).
- **Data export:** CSV file containing computed metrics (`download_metrics_csv()`).

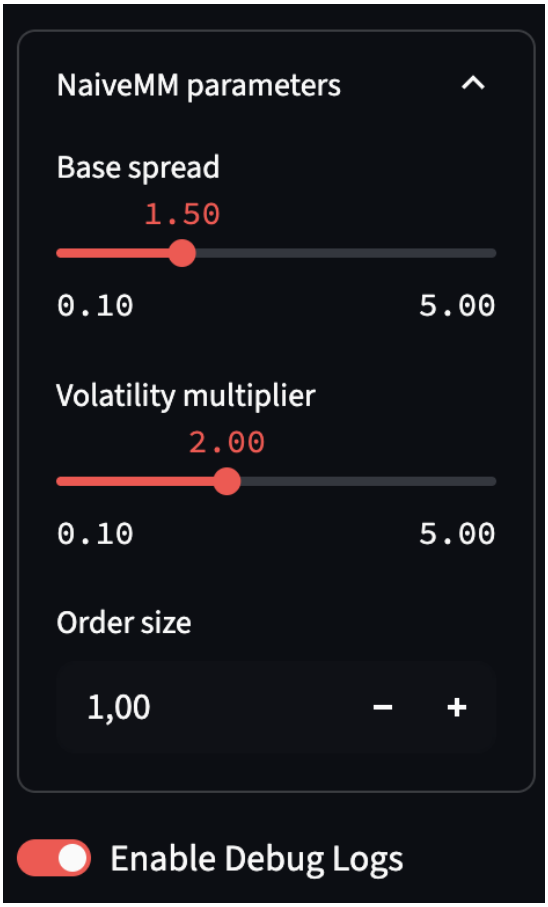
Example Configuration and Results

For demonstration purposes, we configured the simulator to run both the **Stoikov** and **NaiveMM** strategies on the `ETHUSDT` trading pair, using the parameters shown in Figures 7a and 7b. The simulation was executed for a 60-minute data window and performance metrics can be seen in Figures 9–12.

Screenshots of the Interface



(a) Stoikov parameters: risk aversion (γ), market depth (k), order size.



(b) NaiveMM parameters: base spread, volatility multiplier, order size.

Figure 7: Parameter controls for both strategies in the sidebar interface (implemented in `app.py`).

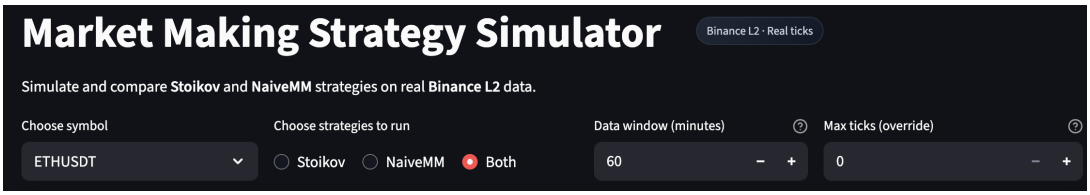


Figure 8: Simulation configuration panel with symbol, strategy choice, data window, and tick limits (bound to `run_strategy()` in `logic.py`).

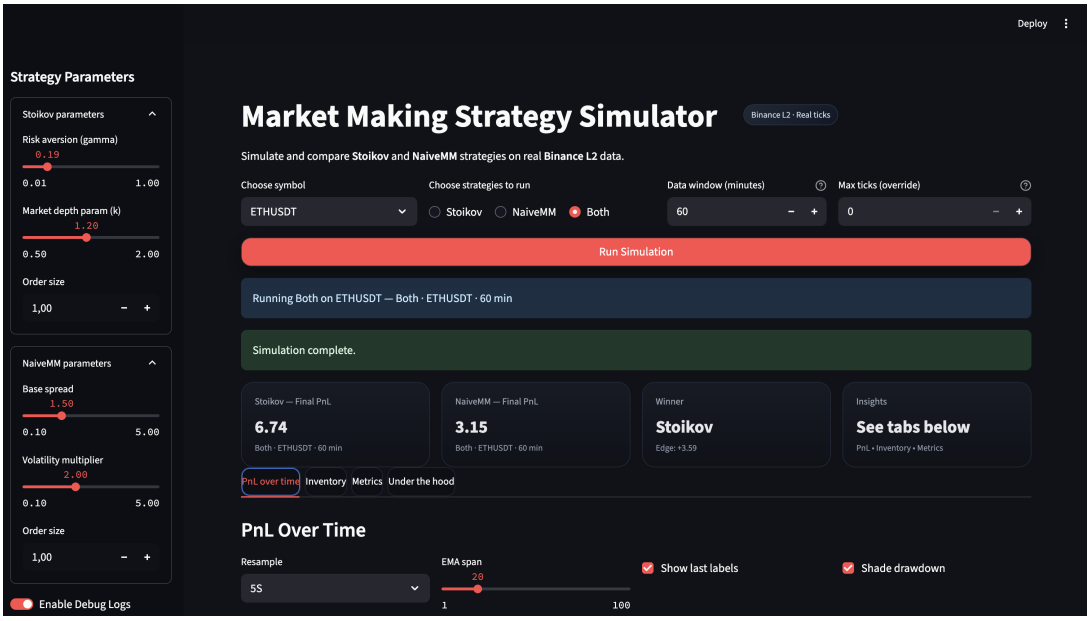


Figure 9: Main dashboard with simulation settings and results summary (tabs renderers from `ui.py`).

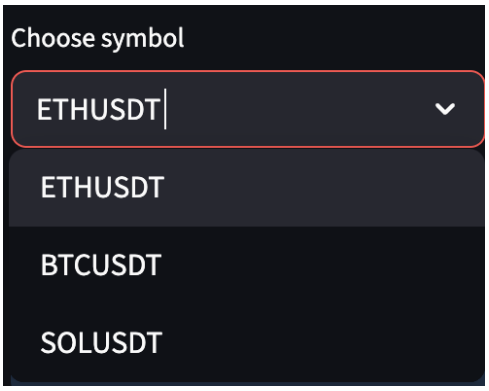


Figure 10: Dropdown menu for selecting a trading symbol (selector in `app.py`).

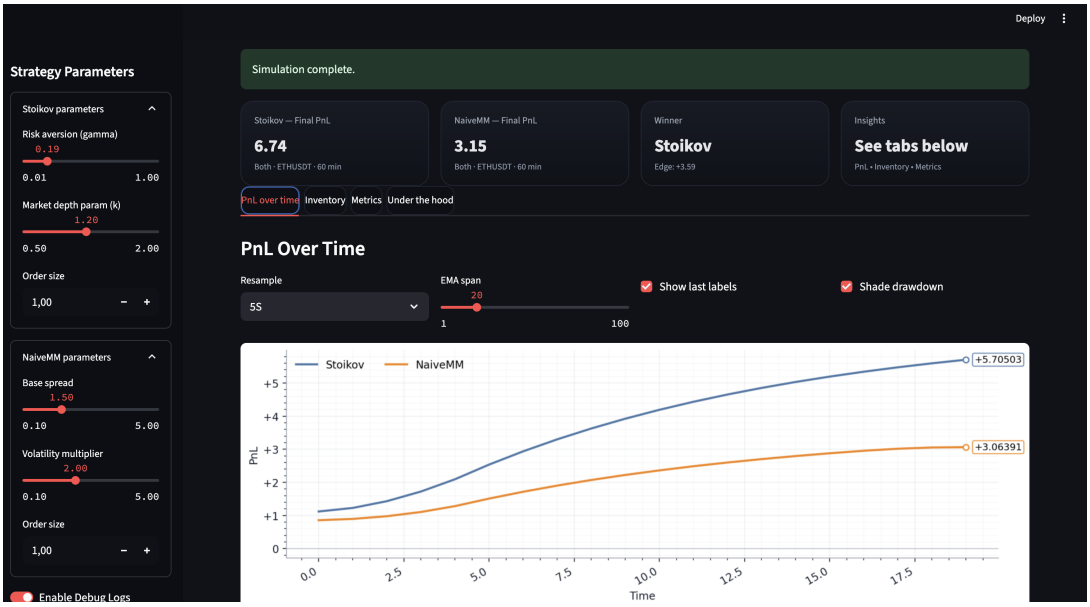


Figure 11: PnL over time comparison between Stoikov and NaiveMM strategies (`show_pnl_plot()` in `ui.py`).

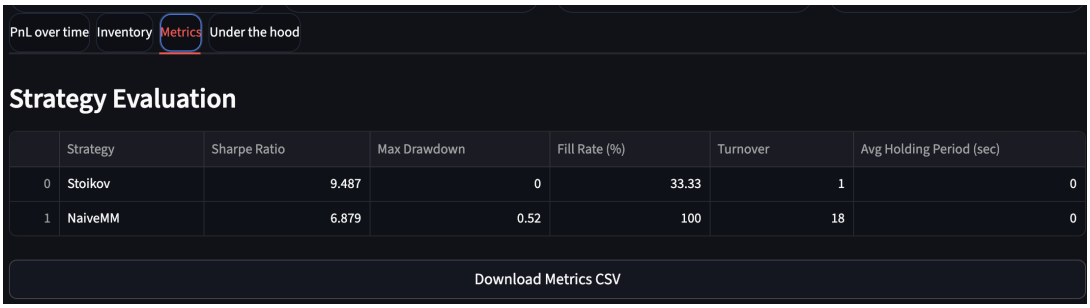


Figure 12: Metrics table with Sharpe ratio, drawdown, fill rate, turnover and holding period (`show_metrics()` fed by `evaluate_strategy_metrics()`).

Full source code for the UI and supporting modules is provided in Appendix.

8 Conclusion

We built a modular market-making simulator on real L2 data combines the Avellaneda-Stoikov model with step-by-step order book reproduction, a simple execution and a pipeline of metrics. All is shown by using `Streamlit` interface, so user can do repeatable runs, see live plots, export results and compare strategies side by side. The system have two example policies—one is inventory-aware Stoikov type, another is basic NaiveMM and it also try to model real spread, delays and fill that happen by probability.

Across case studies on `BTCUSDT` and `SOLUSDT`, run under identical data windows and seeds, Stoikov generated positive cumulative PnL with Sharpe > 1, kept inventory near its target. Relative to NaiveMM, which achieved higher risk-adjusted returns with shallower drawdowns thanks to adaptive spreads and skew. Results are reported using a unified set of diagnostic.

The design is intentionally simple and inspectable: small modules, explicit assumptions and fixed evaluation windows make experiments easy to read, repeat and extend for future updatings. At the same time, some friction factors are modeled in a reduced form (queue position, latency, slippage), which ensures the safety of the system and the possibility of its verification, while maintaining fidelity to the key effects of the microstructure.

Looking ahead, the next natural steps are multi- and cross-asset quoting (e.g., ETH/BTC) with basic arbitrage/hedging hooks, empirical calibration of latency, slippage and queue position from timestamped fills; signal-aware quoting with dynamic parameter tuning via ML; and optional exchange connectors with guardrails (limits, kill-switch, paper/live toggle). In its current form, the simulator is ready for research and teaching, and provides a clear path toward a simple, controlled live setup.

When looking in future, next logical steps are to add multi and cross-asset quoting (like ETH/BTC), with simple arbitrage and hedging function. Also, we can do calibration from real data for latency, slippage and queue position using timestamped fills. Moreover, another idea is signal-aware quoting where parameters are changed by ML in real time. It is also possible to connect exchange with safety features like kill-switch or paper/live switch. Now, simulator is already useful for research and education and it show easy way to go to simple live version with control.

X

References

Avellaneda, M. and Stoikov, S., 2008. High-frequency trading in a limit order book. *Quantitative Finance*, *8*(3), pp.217-224.

Bouchaud, J.-P., Farmer, J.D. and Lillo, F., 2009. How markets slowly digest changes in supply and demand. In *Handbook of financial markets: dynamics and evolution* (pp. 57–160). North-Holland.

Qian, E., Sorensen, E.H. and Hua, R., 2007. Information horizon, portfolio turnover, and optimal alpha models. *The Journal of Portfolio Management*, *34*(1), pp.27–40.

Cartea, Á., Jaimungal, S. and Penalva, J., 2015. *Algorithmic and high-frequency trading*. Cambridge University Press.

Falces Marin, J., Díaz Pardo de Vera, D. and Lopez Gonzalo, E., 2022. A reinforcement learning approach to improve the performance of the Avellaneda–Stoikov market-making algorithm. *PLOS ONE*, *17*(12), p.e0277042.

Li, J., Zhang, J. and Kong, A., 2025. The role of market makers in hybrid markets. *Pacific-Basin Finance Journal*, p.102765.

U.S. Securities and Exchange Commission, 2000. Disclosure of order routing and execution practices. *Proposed Rule 11Ac1-5*, Securities Exchange Act Release No. 43084, 65 FR 48406 (Aug 8, 2000).

Magdon-Ismail, M., Atiya, A.F., Pratap, A. and Abu-Mostafa, Y.S., 2004. On the maximum drawdown of a Brownian motion. *Journal of Applied Probability*, *41*(1), pp.147–161.

Sharpe, W.F., 1994. The sharpe ratio, the journal of portfolio management. *Stanford University, Fall*.

Sortino, F.A. and Price, L.N., 1994. Performance measurement in a downside risk framework. *The Journal of Investing*, *3*(3), pp.59–64.